

# Witbank Elite Car Wash

INT316D — Web Development Group Project

Semester 1, 2026

Individual Reflection Statements

**Student Name:** N.E Rikhotso

**Student ID:** 221398637

**Role:** Frontend Developer (HTML / CSS / JavaScript)

---

## Introduction

Joining the Witbank Elite Car Wash project as the Frontend Developer was both an exciting creative opportunity and a rigorous technical challenge. While I had worked with HTML and CSS in previous modules, this project demanded a level of polish, integration, and responsiveness that I had never attempted before. My role sat at the intersection of design and engineering — I translated the UI/UX Designer's glassmorphism visual language into functional, accessible, Thymeleaf-compatible HTML templates, while implementing the JavaScript logic that made the interface interactive and responsive. This reflection traces my technical journey through the project, the collaboration dynamics I experienced, and the lessons I will carry into my career.

## Technical Contribution and Implementation

My primary responsibility was building the Thymeleaf HTML templates that the Spring Boot backend serves to the browser. Thymeleaf's server-side rendering model requires a different mindset from purely client-side JavaScript frameworks: data binding, conditional rendering, and form submission all happen through Thymeleaf attribute expressions (`th:text`, `th:each`, `th:action`, `th:object`), which meant I needed to understand both the HTML layer and the Spring Model attributes being passed from the backend controllers.

The most complex template I built was the booking form. Implementing Thymeleaf's two-way binding with the Booking model object — using `th:object` and `th:field` — ensured that form data was correctly serialised on submission and that validation errors returned from the controller were displayed inline next to the relevant fields. Getting this right required multiple iterations in collaboration with the Backend Developer, as the model attribute names in the controller had to exactly match the field names in the template.

I implemented all client-side JavaScript using ES6+ syntax without any external libraries — no jQuery, no frameworks. The primary interactive features I built were: the real-time price display that updates when a user selects a service package, the datetime picker validation that prevents bookings in the past, the smooth-scroll navigation between page sections, and the animated confirmation banner that fades in after a successful booking submission. Each feature was written as a self-contained module pattern to avoid global variable pollution.

Achieving full responsiveness across device sizes was the most time-consuming aspect of my work. I used CSS Grid for the page layout structure and Flexbox for component-level arrangements, implementing breakpoints at 320px, 768px, and 1200px. The glassmorphism cards, in particular, required careful testing: backdrop-filter behaves differently across browsers, and I had to implement progressive enhancement so that the blur effect degrades gracefully on browsers that do not support it, falling back to a semi-transparent solid background.

## **Collaboration and Group Dynamics**

My closest working relationship was with the UI/UX Designer, whose CSS variable system I had to adopt and extend throughout all templates. This required a shared understanding of naming conventions and a discipline to never hardcode a colour or spacing value that was already defined as a variable. When I needed a new visual property — such as a hover state colour that was not in the original design — I raised it with the Designer rather than improvising, which kept the visual language coherent across the entire application.

I also interfaced regularly with the Backend Developer to align on the Thymeleaf model attributes. Early in the project, I built several templates against assumed attribute names, which caused integration errors when we first merged our branches on GitHub. This taught me the value of defining interface contracts — in this case, a shared document listing every controller method and its corresponding Model attributes — before building independently.

The Tester's feedback was invaluable for the frontend. Several usability issues were identified — including a form that submitted silently when required fields were empty, and a datetime input that accepted times outside of business hours — that I fixed by adding both HTML5 native validation attributes and custom JavaScript validation logic.

## **Lessons Learned and Future Outlook**

The most important lesson from this project is that frontend development is not decoration — it is engineering. Writing maintainable, accessible, performant HTML, CSS, and JavaScript requires as much architectural discipline as backend development. The decision to use Vanilla CSS3 rather than a framework initially felt like a handicap, but it ultimately forced me to understand the cascade, specificity, and layout algorithms at a level I would never have reached by relying on utility classes.

I also learned that Thymeleaf server-side rendering and client-side JavaScript are complementary, not competing approaches. The server handles data binding and initial page render; JavaScript handles interactivity and progressive enhancement. Understanding this boundary clearly made my code cleaner and more predictable.

In the future I plan to explore React to understand how component-based architecture handles the complexity that grew unwieldy in our Thymeleaf templates as the project expanded. I also want to study web accessibility (WCAG) more formally, having discovered during testing that our booking form had keyboard navigation gaps that affected users who cannot use a mouse.

## **Conclusion**

The Witbank Elite Car Wash project gave me the most complete and demanding frontend challenge of my academic life. I built a product that users could genuinely interact with, that ran on a real server, and that integrated seamlessly with a backend system I did not write.

That integration experience — learning to collaborate across technical boundaries, to communicate through contracts rather than assumptions, and to receive feedback from testers without defensiveness — has prepared me for the collaborative realities of professional software development more than any individual technical skill.